Assignment 4 – Observability

SuiLens Microservices System

Nama : Georgina Elena Shinta Dewi Achti
NPM : 2206810995
Mata Kuliah : Arsitektur Aplikasi Web A

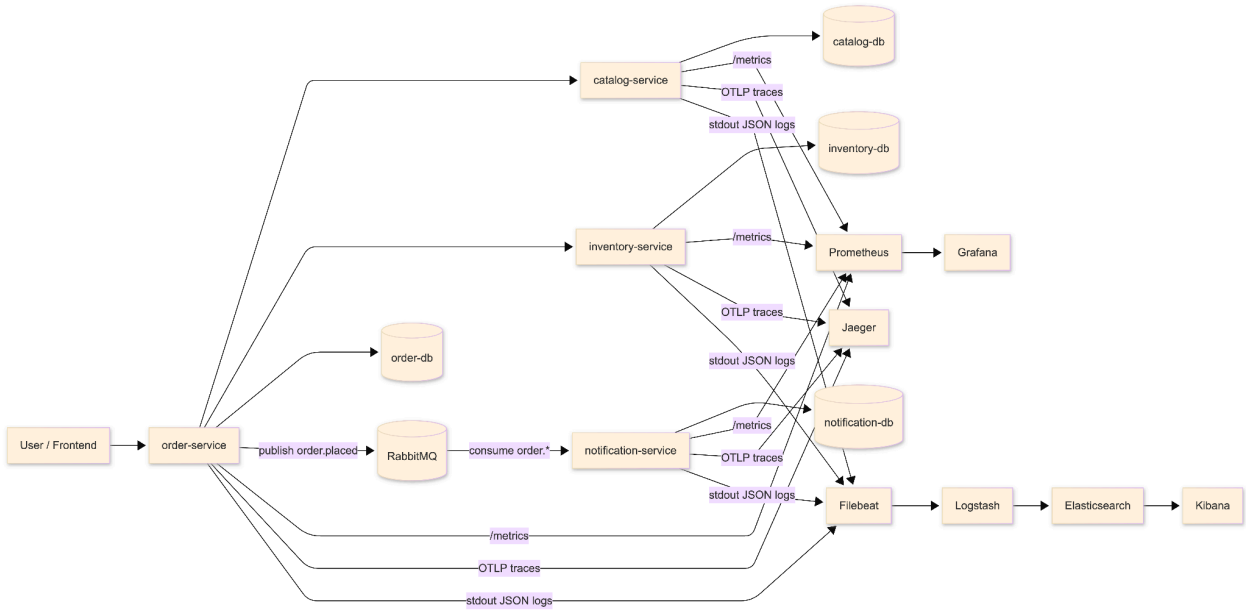
1. Pendahuluan

Pada tugas ini, saya mengimplementasikan observability pada sistem microservices SuiLens. Observability menjadi komponen penting karena sistem terdiri dari beberapa service yang saling berinteraksi, sehingga diperlukan mekanisme untuk memantau performa, mengumpulkan log secara terpusat, serta menelusuri alur request secara end-to-end.

Fokus implementasi mencakup tiga aspek utama, yaitu monitoring, centralized logging, dan distributed tracing. Ketiga aspek ini bertujuan untuk memberikan visibilitas yang menyeluruh terhadap kondisi sistem, baik dari sisi teknis maupun proses bisnis.

2. Gambaran Solusi dan Arsitektur

Dalam implementasi ini, saya menggunakan beberapa tools utama yang masing-masing berperan pada aspek observability yang berbeda. Untuk monitoring digunakan Prometheus sebagai pengumpul metrik dan Grafana sebagai dashboard visual. Untuk logging digunakan ELK stack yang terdiri dari Filebeat, Logstash, Elasticsearch, dan Kibana. Sedangkan untuk tracing digunakan OpenTelemetry dengan Jaeger sebagai visualisasi trace.



Secara arsitektur, setiap service menghasilkan data observability melalui mekanisme yang berbeda. Metrics diambil dari endpoint `/metrics` dan dikumpulkan oleh Prometheus untuk kemudian divisualisasikan di Grafana. Log dihasilkan dalam format structured JSON dan dikirim ke stdout, kemudian dikumpulkan oleh Filebeat, diproses oleh Logstash, dan disimpan di Elasticsearch sebelum

ditampilkan di Kibana. Sementara itu, trace dikirim menggunakan OpenTelemetry dan divisualisasikan melalui Jaeger. Diagram arsitektur yang disertakan menunjukkan bagaimana seluruh komponen tersebut terintegrasi dalam satu sistem observability.

3. Langkah Implementasi

Monitoring

Pada bagian monitoring, saya menambahkan endpoint `/metrics` pada setiap service backend. Endpoint ini menyediakan metrik yang kemudian dikumpulkan oleh Prometheus. Metrik yang diimplementasikan meliputi jumlah request, latency request, error rate, serta metrik bisnis.

Jumlah request digunakan untuk melihat volume traffic pada sistem. Latency diukur menggunakan histogram dan ditampilkan dalam bentuk p95 untuk merepresentasikan performa mayoritas request. Error rate dihitung berdasarkan proporsi request dengan status 5xx terhadap total request. Selain itu, metrik bisnis seperti jumlah order, reservasi inventory, dan notifikasi ditambahkan untuk memberikan gambaran aktivitas sistem dari sisi bisnis.

Seluruh metrik tersebut divisualisasikan dalam dashboard Grafana sehingga kondisi sistem dapat dipantau secara real-time.

Centralized Logging

Untuk centralized logging, saya menerapkan structured logging pada setiap service backend. Log dihasilkan dalam format JSON agar mudah diproses dan dianalisis. Setiap log memuat informasi penting seperti timestamp, nama service, level log, message, endpoint, correlation_id, dan trace_id.

Log yang dihasilkan oleh service dikirim ke stdout container, kemudian dikumpulkan oleh Filebeat. Selanjutnya, log diproses oleh Logstash sebelum disimpan di Elasticsearch. Kibana digunakan untuk mengakses dan memvisualisasikan log tersebut dalam satu tempat.

Dengan pendekatan ini, log dari berbagai service dapat dianalisis secara terpusat. Correlation_id digunakan untuk menelusuri satu request yang sama di berbagai service sehingga mempermudah proses investigasi.

Distributed Tracing

Pada bagian tracing, saya menggunakan OpenTelemetry untuk meng instrumentasi service backend. Tracing dilakukan pada inbound request, outbound call antar service, serta proses publish dan consume event melalui RabbitMQ. Flow utama yang dianalisis adalah proses create order. Request dimulai dari order-service, kemudian dilanjutkan dengan validasi ke catalog-service dan reservasi ke inventory-service. Setelah itu, event dikirim ke RabbitMQ dan diproses oleh notification-service.

Trace yang dihasilkan dikirim ke Jaeger, yang menampilkan hubungan antar span serta durasi setiap proses. Dengan visualisasi ini, alur request dapat dipahami secara menyeluruh dan bottleneck dalam sistem dapat diidentifikasi dengan lebih mudah.

4. Cara Menjalankan Sistem

Sistem dijalankan menggunakan Docker Compose dengan perintah berikut:

```
docker compose up --build
```

Setelah seluruh service berjalan, masing-masing tools dapat diakses melalui URL berikut:

```
Grafana: http://localhost:3005
Kibana: http://localhost:5602
Jaeger: http://localhost:16686
API (Order Service): http://localhost:3002/api/orders
API (Catalog Service): http://localhost:3001/api/lenses
```

Untuk menghasilkan data observability, dilakukan request ke endpoint create order dengan menyertakan header x-correlation-id. Request ini akan memicu munculnya metrics, log, dan trace sehingga dapat diamati pada masing-masing tools.

Contoh request yang digunakan:

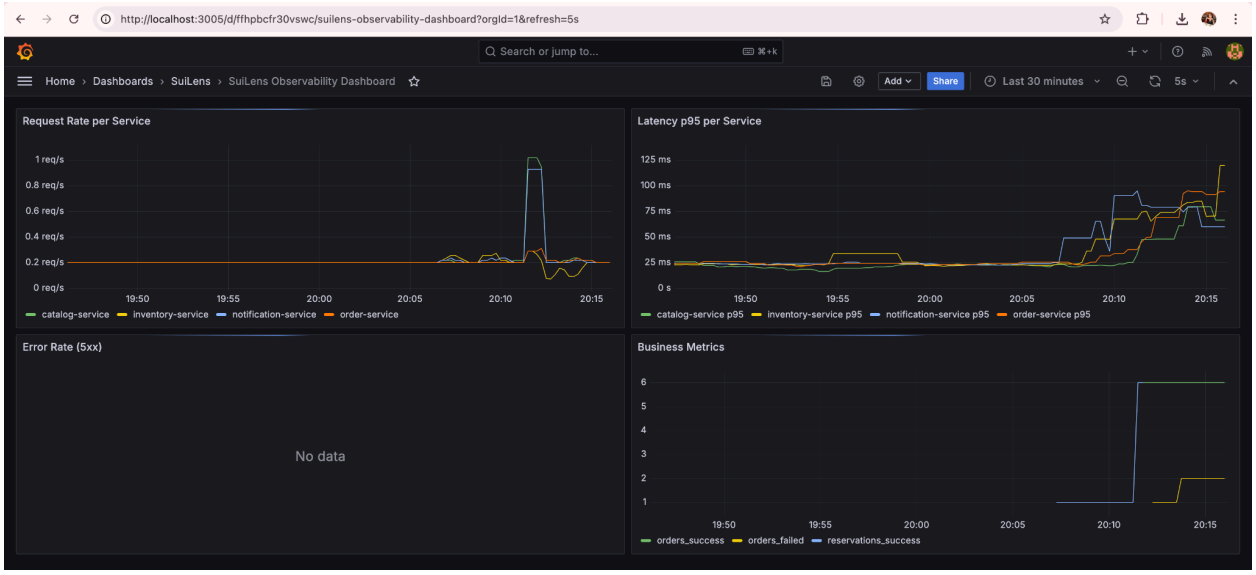
```
curl -X POST http://localhost:3002/api/orders

-H "Content-Type: application/json"
-H "x-correlation-id: test-123"
-d "{
"customerName": "Test User",
"customerEmail": "test@test.com
",
"lensId": "<lens_id>",
"branchCode": "KB-JKT-S",
"startDate": "2026-04-01",
"endDate": "2026-04-03"
}"
```

5. Hasil Implementasi

Monitoring

Hasil monitoring dapat diamati melalui dashboard Grafana yang menampilkan berbagai metrik utama sistem secara real-time. Berdasarkan hasil pengujian yang dilakukan, grafik request rate menunjukkan adanya aktivitas request yang meningkat setelah dilakukan load testing. Hal ini menandakan bahwa sistem berhasil menerima dan memproses request sesuai skenario yang dijalankan.

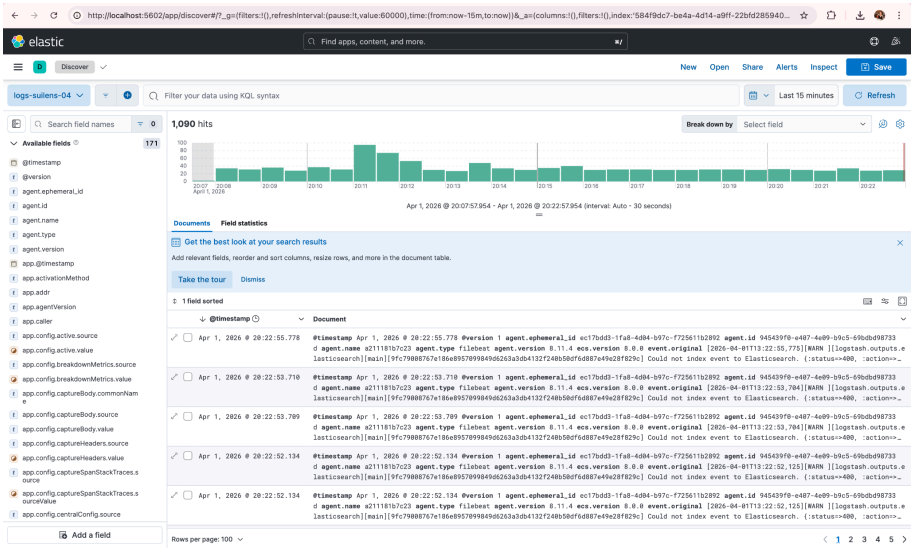
Selain itu, latency p95 menunjukkan waktu respon mayoritas request yang relatif stabil, sehingga dapat disimpulkan bahwa performa sistem masih berada dalam kondisi yang baik. Metrik bisnis seperti jumlah order dan reservasi juga mengalami peningkatan, yang menunjukkan bahwa proses bisnis utama berjalan dengan sukses. Secara keseluruhan, dashboard Grafana memberikan gambaran yang jelas mengenai kondisi sistem selama pengujian berlangsung.

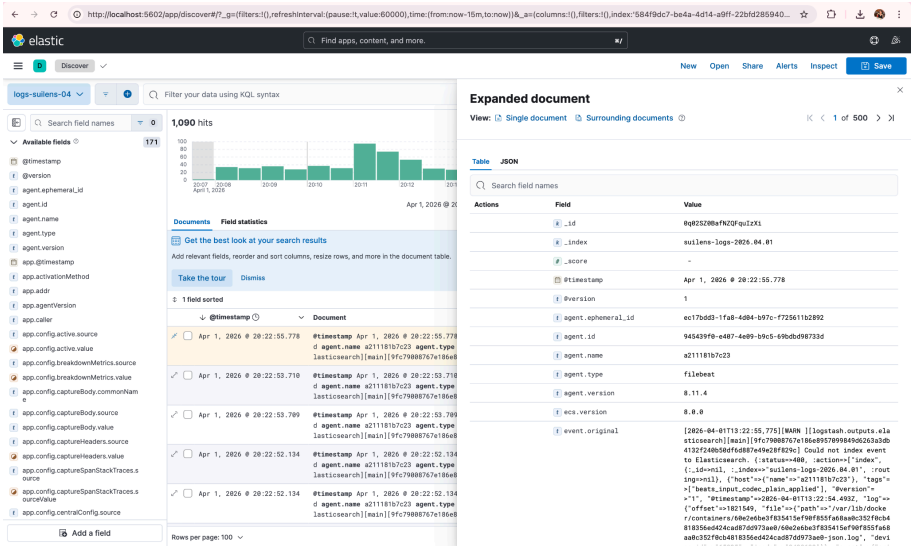


Centralized Logging

Pada bagian centralized logging, hasil implementasi dapat dilihat melalui Kibana yang menampilkan log dari berbagai service dalam satu tampilan terpusat. Berdasarkan screenshot yang ditampilkan, log berhasil dikumpulkan dan ditampilkan dalam format structured JSON, sehingga informasi seperti timestamp, agent, dan isi pesan log dapat dianalisis dengan mudah.

Selain itu, dengan membuka detail log (expanded view), terlihat bahwa setiap log memuat informasi yang cukup untuk kebutuhan investigasi. Hal ini mempermudah proses debugging karena setiap kejadian dalam sistem dapat ditelusuri secara lebih jelas. Dengan adanya pendekatan centralized logging ini, analisis terhadap error maupun aktivitas sistem dapat dilakukan tanpa harus mengakses log dari masing-masing service secara terpisah.

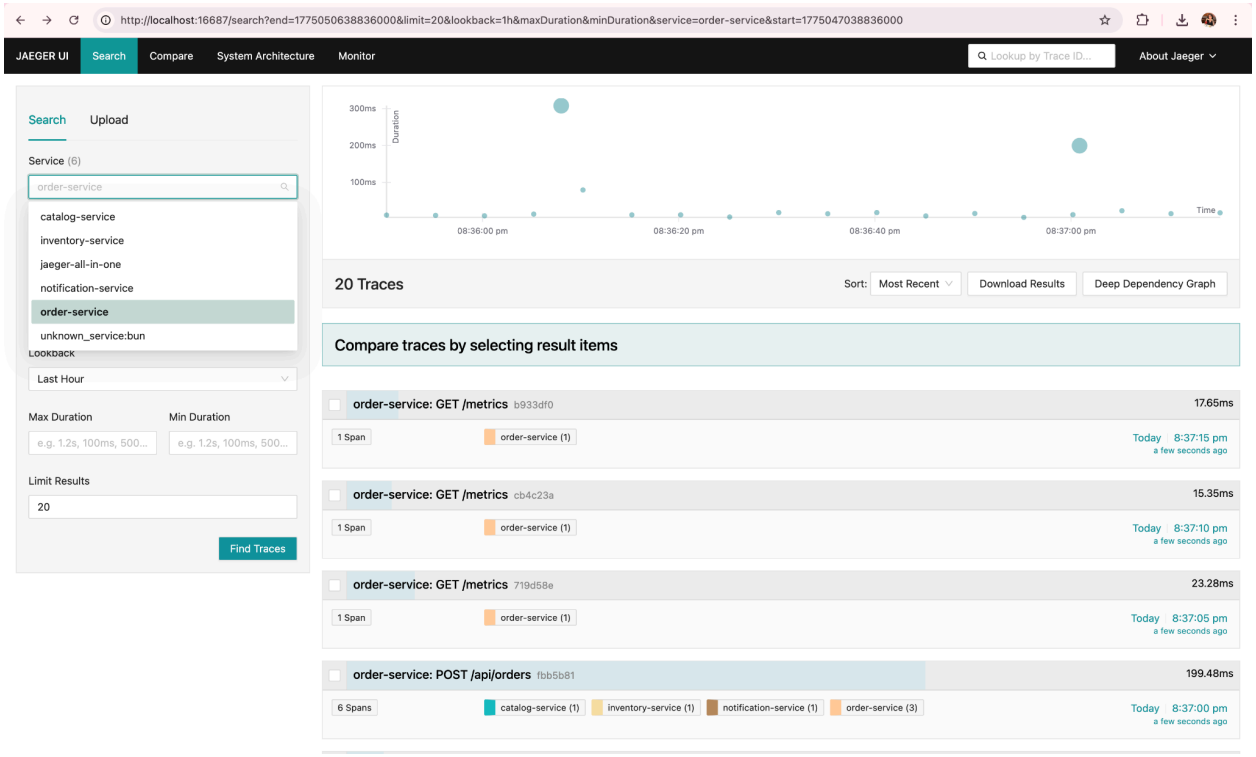




Distributed Tracing

Pada bagian distributed tracing, hasil implementasi dapat dilihat melalui Jaeger yang menampilkan trace dari alur request dalam sistem. Berdasarkan hasil yang ditampilkan, trace untuk proses create order berhasil direkam dan divisualisasikan.

Trace tersebut menunjukkan bahwa request dimulai dari order-service, kemudian dilanjutkan ke service lain seperti catalog-service dan inventory-service, serta diproses lebih lanjut oleh notification-service. Setiap tahapan direpresentasikan dalam bentuk span yang memiliki durasi eksekusi.



Dengan adanya visualisasi ini, hubungan antar service dapat dipahami dengan lebih jelas. Selain itu, durasi pada setiap span memungkinkan identifikasi terhadap bagian sistem yang memiliki waktu proses lebih lama, sehingga potensi bottleneck dapat dianalisis secara lebih efektif.

5. Penjelasan Metrik, Log, dan Flow Trace

Pada bagian monitoring, metrik yang ditampilkan pada dashboard Grafana menunjukkan aktivitas sistem secara real-time. Grafik request rate memperlihatkan jumlah request yang masuk ke sistem, di mana peningkatan nilai menunjukkan adanya peningkatan traffic. Selain itu, latency p95 digunakan untuk merepresentasikan waktu respon mayoritas request, sehingga dapat memberikan gambaran performa sistem secara umum. Peningkatan latency pada waktu tertentu dapat mengindikasikan adanya bottleneck pada salah satu service. Metrik bisnis seperti jumlah order dan reservasi juga menunjukkan bahwa sistem berjalan sesuai dengan proses bisnis yang diharapkan.

Pada bagian centralized logging, log yang ditampilkan pada Kibana menunjukkan data yang telah dikumpulkan dari berbagai service dalam satu tempat. Log disajikan dalam format structured JSON sehingga setiap informasi seperti timestamp, agent, dan message dapat dibaca dengan jelas. Dengan adanya informasi ini, proses debugging menjadi lebih mudah karena dapat diketahui kapan suatu event terjadi dan apa yang menyebabkan error. Selain itu, penggunaan correlation_id memungkinkan pelacakan satu request yang sama di berbagai service.

Pada bagian distributed tracing, visualisasi trace pada Jaeger menunjukkan alur request secara end-to-end. Proses dimulai dari order-service sebagai entry point, kemudian dilanjutkan ke catalog-service untuk validasi, serta inventory-service untuk reservasi. Setelah itu, proses dilanjutkan melalui mekanisme event-driven yang diproses oleh notification-service. Setiap proses direpresentasikan dalam bentuk span yang memiliki durasi eksekusi, sehingga dapat digunakan untuk mengidentifikasi bagian sistem yang membutuhkan waktu paling lama dan berpotensi menjadi bottleneck.